## Where to Use MongoDB:

- To store large amount of Data
- Mobile and Web Applications
- User Data Management
- When there is no need of schema.

## Advantages of MongoDB over RDBMS:

- Schema less
- Structure of a single object is clear.
- No complex queries like joins.
- Extremely faster than RDBMS.

**C-CREATE**
**R-READ**
**U-UPDATE**
**D-DELETE**

## CREATE

db.version()------->To know the version of database

To know current database ------ db

To See Databases: show dbs or show databases.

### To Create or switch database in mongodb

**Syntax:** use DatabaseName

- Ex: use MyFirstProject

### To Create Collection:

**Syntax:**

Explicitly: db.createCollection('collection_Name')

Implicitly: db.collectionName.insertOne({key:'value'})

**To See all Collections:**

- show collections

**To Delete or Drop collection in mongodb:**

- db.collectionName.drop()

  Ex: db.Project1.drop()

**To Delete or Drop database in mongodb:**

- db.databaseName.drop()

  Ex: db.MyFirstProject.drop()

**To Delete or Drop current database:**

- db.dropDatabase()

**TO INSERT THE FILEDS INTO THE COLLECTION:**

1)insertOne()

2)insertMany()

## insertOne() :-

- This function is used to insert one document into the collection.
- It accepts Object

**Syntax:**

db.collectionName.insertOne();

Ex: db.trainers.insertOne({name : “Ur_Wish”}).

## insertMany():

- This function is used to insert Multiple documents at a time .
- To insert multiple documents we need to use Arrays

**Syntax:**

db.collectionName.insertMany([{},{},{},----------])

Ex:

db.trainers.insertMany([

{name :"neeraj",course:"mongodb",duration:"1month",price:20000},

{name :“pavan",course:"sql",duration:"1month",price:30000}

])

## Collection:

- Collection is a group of MongoDB documents.
- It is the equivalent of an RDBMS table.
- A collection exists within database.
- Collections do not have a particular schema.
- Documents within a collection can have different fields.

## Document:

- A document is a set of key-value pairs.
- Representation of document is curly braces {}.
- Documents have dynamic schema.
- Common fields in the documents may hold different types of data.
- Maximum size of each document is 16mb.

## CAPPED COLLECTION:

- When we want to restrict the size of collection then we go for capped collection.
- Fixed size Collection are called as capped collection in mongoDB.
- While creating a collection user must specify the collecton’s max size in bytes and max number of documents to store.

**Ex:**

db.createCollection(“mytrainers”,{capped : true ,size :10000,max:2 })

To check wheather Capped_Collection is created or not:

Syntax: db.collectionName.isCapped()

## Objectid:

- ObjectId will be generated by default at the time of field insertion.
- ObjectId is unique and it acts like a primary key for the document.
- It is a 12-byte BSON type hexadecimal string.
- Ex:
    - ObjectId("650e81b4f36d312bcd4032b4")
    - 4 bytes: TimeStamp
    - 5 bytes: Random_Value
    - 3 bytes: Increment

### READ

To read the documents in mongodb we have 2 methods

**Find() :**

- It is a primary method for retrieving documents from a collection in MongoDB.
- This method accepts a query object. If left empty, all documents will be returned.
- If no matching document is found, it returns null.

**Syntax: db.collection.find({query}, {projection})**

- **query**: This is the filter criteria to specify which documents you want to retrieve.
- **projection**: This defines which fields to include or exclude from the results. If not specified, all fields will be returned by default

**Ex:** To retrieve all the documents

**db.student.find({}) or db.student.find()**

To retrieve documents where age is 18

**db.student.find({age:18})**

**findOne()**

- To select only one document, we can use the findOne() method.
- This method accepts a query object. If left empty, it will return the first document it finds in the collection.

- It returns only one document, even if multiple documents match the criteria.
- If no matching document is found, it returns null.

**Syntax: db.collection.findOne({query}, {projection})**

- **query:**
- **projection:**

**Ex:** To retrieve first document in collection

**db.student.findOne({}) or db.student.findOne()**

To retrieve the first matching document where age is 18

**db.student.findOne({age:18})**

**Operators:**

Operators are special symbols which are used to perform specific tasks between two or more operands.

## Query Operators:

Query Operators in MongoDB are used to filter documents based on some specific criteria within their fields.

**$eq:** Matches the values of the fields that are equal to a specified value.

**Syntax: { field : { $eq : value } }  or { field : value }**

**Ex:** db.emp.find({ gender : { $eq : Male }})

**$ne:** Matches all values of the field that are not equal to a specified value.

db.emp.find({ gender : { $ne : Male }})

**$gt:** Matches values of the fields that are greater than a specified value.

db.emp.find({ age : { $gt : 20 }})

**$gte:** Matches values of the fields that are greater than equal to the specified value.

```
db.emp.find({ age : { $gte : 20 }})
```

**$lt:** Matches values of the fields that are less than a specified value

```
db.emp.find({ age : { $lt : 20 }})
```

**$lte:** Matches values of the fields that are less than equal to the specified value

```
db.emp.find({ age : { $lte : 20 }})
```

**$in:** Matches any of the values specified in an array.

**Syntax: { field: { $in: [<value1>, <value2>, ... <valueN> ] } }**

Ex: db.emp.find( { sal: { $in: [ 5000, 1500 ] } })

**$nin:** Matches none of the values specified in an array.

**Syntax: { field: { $nin: [<value1>, <value2>, ... <valueN> ] } }**

Ex: db.emp.find( { sal: { $nin: [ 5000, 1500 ] } })

## Logical Operators:

**$and:**

- It is used to perform logical AND operation of array of one or more expressions.
- All the conditions should be true.

**Syntax: { $and: [ { Expression 1 }, { Expression 2 }, { Expression 3 }, ..., { Expression N } ]}**

**or**

**{ { Expression 1 }, { Expression 2 }, { Expression 3 },..., { Expression N }}**

### $or:

- It is one of the logical operators of an array of one or more expressions.
- The $or operator performs a logical "OR operation" on an array of one or more <expressions> and selects the documents that satisfy at least one of the <expressions>.
- Any one condition is satisfied that's enough.

**Syntax: { $or: [ { <expression1> }, { <expression2> }, ... , { <expressionN> } ] }**

### $nor:

- The $nor operator is used to perform logical "NOR operations" on an array of one or more expressions.
- This operator is also used to select or retrieve documents that do not match all of the given expressions in the array.

**Syntax: { $nor: [ { Expression1 }, { Expression2 }, ... { ExpressionN } ] }**

### $not:

- The $not operator is used to perform logical "NOT operation" on the specified expressions.
- You can use it with any other query expression, such as equal to ($eq), greater than ($gt), or less than ($lt).
- It selects or retrieves only those documents that do not match the given expression.

**Syntax: { field: { $not: { operator-expression } } }**

## Element Query Operators:

Element operators return data based on field existence or data types.

### $exists:

- The $exists operator matches documents that contain or do not contain a specified field, including documents where the field value is null.

  **Syntax: { field: { $exists: <boolean> } }**

### $type:

- $type selects documents where the value of the field is an instance of the specified BSON type(s).
- Querying by data type is useful when dealing with highly unstructured data where data types are not predictable.

**Syntax: { field: { $type: <BSON type> } }**

## Evaluation Operators:

Evaluation operators return data based on evaluations of either individual fields or the entire collection's documents.

### $expr:

- The $expr operator is used to allow the use of aggregation expressions within query operations.
- You can compare two fields in the same document.